DIGITAL IC

HIGH PERFORMANCE FPU OPTIMIZED FOR RISC-V

Mohamed Gamal

March 2026.

Table of Contents

1. Introduction

Floating-point arithmetic plays a critical role in modern processors, enabling efficient computation for scientific applications, signal processing, graphics, and machine learning workloads. The IEEE-754 standard defines a widely adopted representation and behavior for floating-point numbers, ensuring numerical consistency across hardware and software platforms.

In many processor architectures, the Floating-Point Unit (FPU) is responsible for implementing arithmetic operations such as addition, multiplication, division, and square root according to the IEEE-754 specification. While basic implementations of these units are relatively straightforward, achieving high performance requires careful algorithm selection and hardware optimization.

This project focuses on the design and optimization of an IEEE-754 single-precision floating-point unit integrated into a custom RV32IF RISC-V processor. Several arithmetic modules were redesigned to improve performance by replacing sequential algorithms with parallel arithmetic approaches more suitable for FPGA architectures.

In particular, the square root unit was redesigned using a reciprocal square-root iteration implemented in fixed-point arithmetic, significantly reducing latency. The divider architecture was optimized using the Goldschmidt division algorithm, which replaces sequential subtraction operations with parallel multiplier-based refinements. Additionally, the floating-point adder was improved by replacing iterative alignment and normalization loops with a barrel shifter and leading-zero counter.

The optimized implementations were synthesized and implemented using Xilinx Vivado, allowing the analysis of resource utilization, timing behavior, and architectural trade-offs. The results demonstrate how algorithmic and architectural improvements can significantly reduce floating-point operation latency while maintaining correct IEEE-754 behavior.

2. Related Work

Floating-point units (FPUs) are commonly implemented using iterative arithmetic algorithms due to their relatively low hardware cost. Several open-source implementations exist, such as Dawson's IEEE-754 compliant FPU and Berkeley's HardFloat library. Many baseline implementations rely on restoring division, Newton–Raphson iterations, or sequential normalization loops.

While these approaches are simple and hardware efficient, they often introduce high latency due to sequential operations. In FPGA-based systems, multiplier resources (DSP blocks) are

widely available and can be leveraged to accelerate arithmetic operations through parallel computation.

This project explores architectural optimizations that replace sequential arithmetic with parallel multiplier-based algorithms. Goldschmidt division and reciprocal square-root iterations are used to significantly reduce operation latency while maintaining IEEE-754 style design. The impact of these optimizations is evaluated through FPGA synthesis and implementation results.

3. IEEE 754 Single-Precision — The Common Foundation

All modules encode and decode numbers in IEEE 754-2008 binary32 format. Every algorithm in this FPU starts by unpacking this format and ends by packing a result back into it.

Field	Bits	Width	Meaning
Sign (S)	bit 31	1 bit	0 = positive, 1 = negative
Exponent (E)	30–23	8 bits	Biased by 127; actual exponent = E − 127
Mantissa (M)	22–0	23 bits	Fractional part. Normal numbers have implicit leading 1

3.1. Guard / Round / Sticky Rounding — Used by Every Module

After every arithmetic operation, the result must be rounded to fit back into 23 mantissa bits. All three versions of all four modules use the same Round-to-Nearest-Even scheme with three extra bits tracked during computation:

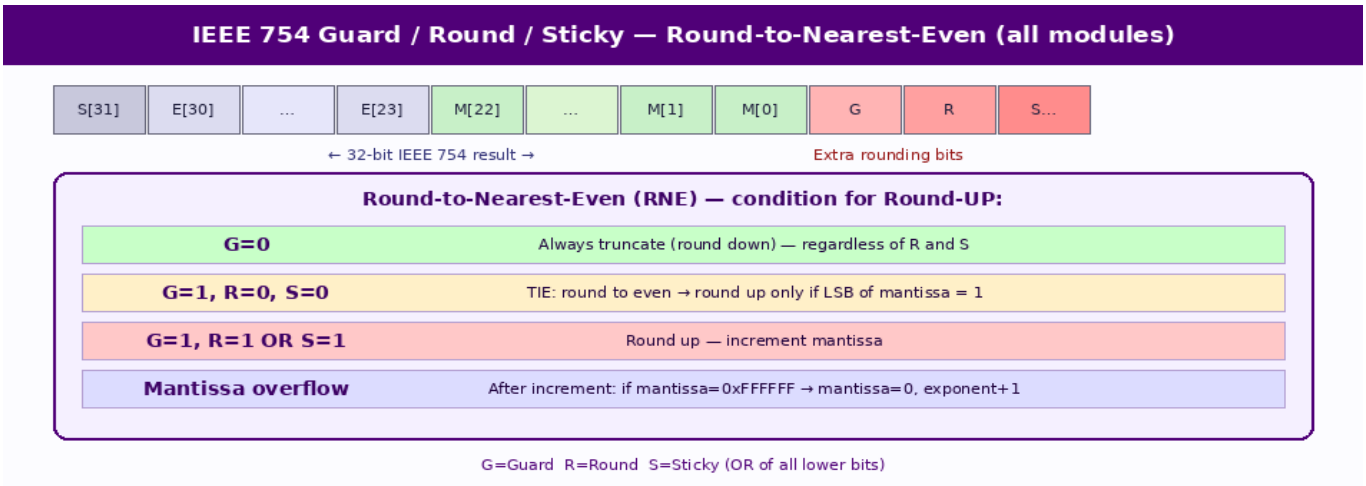


Figure: Guard / Round / Sticky bits — IEEE 754 Round-to-Nearest-Even rule

Guard (G): The bit immediately below the stored LSB.

Round (R): One bit further below Guard.

Sticky (S): Logical OR of ALL remaining lower bits. Once set to 1, it stays 1 through any further shifts — it 'remembers' whether any non-zero bits were discarded.

Round UP when: G = 1 AND (R = 1 OR S = 1 OR LSB = 1)

4. Floating-Point Square Root — Largest Speedup (30×)

Square Root Architecture

The baseline square root implementation used a Newton iteration operating entirely in floating-point format. In this approach, each iteration required floating-point division, which significantly increased the computational latency. In practice, this resulted in very high cycle counts due to the cost of the floating-point divider.

To improve performance, the square root unit was redesigned using a reciprocal square-root iteration. Instead of computing the square root directly, the algorithm computes an approximation of the reciprocal square root:

$$y = \frac{1}{\sqrt{x}}$$

The Newton iteration for reciprocal square root is given by:

$$y_{n+1} = y_n \left(\frac{3}{2} - \frac{x}{2} y_n^2\right)$$

Once the reciprocal square root is computed, the final square root is obtained by multiplying the result with the original operand.

To further improve efficiency, the iteration was implemented using fixed-point arithmetic (Q24.24 format). This eliminates repeated floating-point unpacking and normalization steps, significantly reducing control complexity and register usage.

An initial approximation is generated using a small lookup table based on the leading bits of the mantissa. This initial guess allows the iteration to converge rapidly within a small number of steps.

4.1. v0 (Dawson): No Dedicated sqrt Module

The original Dawson FPU project does not include a square root module. The divider_old.v used in this project is the v1 upgraded divider, not Dawson's original. The sqrt capability was added from scratch in this project.

4.2. v1 : Newton-Raphson via FPU Submodule Calls

The Newton-Raphson Formula

Newton-Raphson iteratively refines a guess y toward √x by solving $f(y) = y^2 - x = 0$:

$$y_{n+1} = 0.5 \times (y_n + x / y_n)$$

This converges quadratically — each iteration roughly doubles the number of correct digits. Starting from a good initial guess, 4 iterations give single-precision accuracy.

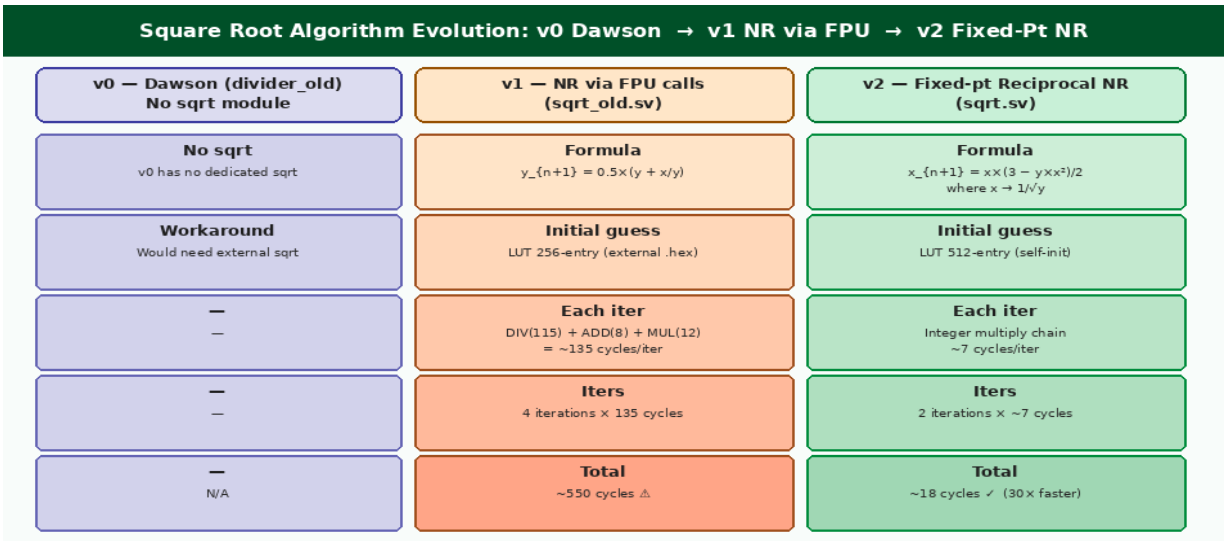


Figure: Square root algorithm across three versions

v1 Implementation: Chained FPU Modules

Architecture: sqrt_old.sv instantiates the project's own divider1 (v1 divider), adder, and multiplier modules as submodules. The FSM issues start pulses and waits for done signals from each submodule.

Step	FPU call	Cycles	Purpose
Init	sqrt_lut (combinational)	0 cycles	Get initial guess from 256-entry hex LUT
①×4	divider1	~115 cycles each	Compute x / y_n
②×4	adder	~8 cycles each	Compute y_n + (x/y_n)
③×4	multiplier × 0.5	~12 cycles each	Compute result × 0.5
Per iteration	—	~135 cycles	div + add + mul overhead
×4 iterations	—	~540 cycles	Full Newton-Raphson

Why so slow? Each of the three FPU submodules has its own FSM. The sqrt module must: assert start → wait multiple cycles → receive done → assert next start. The divider alone costs 115 cycles × 4 iterations = 460 cycles, dwarfing everything else. The overhead of crossing module boundaries is enormous.

4.3. v2 (sqrt.sv): Fixed-Point Newton-Raphson on Reciprocal Square Root

The Key Algorithmic Shift: Compute 1/√x instead of √x

The Newton-Raphson update for finding $x = 1/\sqrt{y}$ (i.e., solving $g(x) = 1/x^2 - y = 0$) is:

$$x_{n+1} = x_n \times (3 - y \times x_n^2) \;/\; 2$$

This is the **fast inverse square root** formula. The final result is recovered as $\sqrt{y} = y \times (1/\sqrt{y})$.

Why is this better? Each iteration is three integer multiplications and a subtraction — no floating-point normalization, no FSM state transitions, no module boundary overhead. The entire computation runs sequentially inside a single always block in Q24.24 fixed-point arithmetic (48-bit registers where bit 24 = 1.0).

Q24.24 Fixed-Point Arithmetic

All values are stored in 48-bit integers with an implicit binary point after bit 24. Multiplication is: result = (a × b) >> 24. This lets the synthesis tool implement everything as simple integer multipliers — no IEEE 754 decoding/encoding between steps.

512-Entry Self-Initializing LUT

The v2 LUT has **512 entries** (vs 256 in v1), indexed by mant_in[22:14] — 9 bits instead of 8. More entries → better initial approximation (~9 bits instead of ~8) → fewer Newton-Raphson iterations needed. The LUT is generated at elaboration time by a SystemVerilog initial block using $sqrt() — no external .hex file required.

Iteration Count: Why 2 Instead of 4

Iteration	v1 (FP-level NR on √x)	v2 (fixed-pt NR on 1/√x)
Start	8-bit LUT initial guess	9-bit LUT initial guess
After iter 1	~16 bits accurate	~18 bits accurate
After iter 2	~24 bits ✓	~24 bits ✓ — DONE
Iters 3–4	Needed (FP overhead per iter)	NOT NEEDED
Cost per iter	~135 cycles (FPU calls)	~7 cycles (integer ops)

4.4. Cycle Count — Square Root

Phase	v1 cycles	v2 cycles
Special cases	~3 cycles	2 cycles
LUT lookup	Combinational (0)	1 cycle (INIT)
Newton-Raphson iteration 1	~135 cycles	6 cycles
Newton-Raphson iteration 2	~135 cycles	8 cycles
Iterations 3–4	~270 cycles	NOT NEEDED
Final extraction	~1 cycle	2 cycles
TOTAL	~550 cycles	~18 cycles

Speedup	—	≈ 30×

COMPOUNDING SPEEDUP FACTORS: (1) Fixed-point integer iterations cost ~7 cycles vs ~135 cycles for FPU-level iterations — 19× per iteration. (2) Only 2 iterations are needed instead of 4 — 2× reduction. Combined: ~30× total speedup. The architectural change (eliminating three FPU submodule instantiations) is what enables both factors simultaneously.

4.5. FPGA Implementation Results — Vivado Post-Implementation

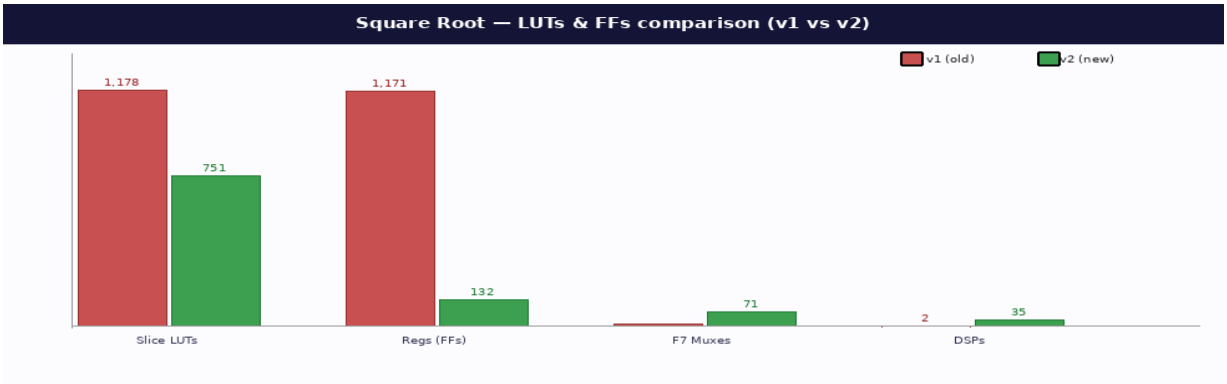
All modules were synthesised and implemented in Vivado **for Arty A7-35 (xc7a35tcsg324-1, Artix-7)**. Results below are post-implementation (not post-synthesis) and reflect real routed timing and utilisation.

v1 Algorithm

Resource	v1 value	Why so large
Slice LUTs	1178	Three complete FPU submodule FSMs synthesised together
Slice Registers	1171	Every submodule has its own full register set + pipeline regs
DSPs	2	Only adder/multiplier use small 24-bit DSPs
WNS	10.679 ns	Critical path is simple FSM transitions — very short
Max Freq	~175 MHz	Could run much faster — bottleneck is cycle count, not timing
Wall time	~9,900 ns	550 cycles × 18 ns = massive latency

v2 Algorithm

Resource	v2 value	Why
Slice LUTs	751 (−36%)	No submodule FSMs; just control logic + mux trees
Slice Registers	132 (−89%)	Single working register set; no chained FSM state
F7 Muxes	71	Wide mux trees from NR correction factor logic
DSPs	35 (+33)	48-bit Q24.24 multipliers (x×x, y×x², x×temp) → DSP48 chains
WNS	0.488 ns	DSP cascade = long combinational path; 18ns budget mostly used
Max Freq	~57 MHz	DSP chain limits frequency — need wider clock period
Wall time	~324 ns (−97%)	18 cycles × 18 ns



5. Floating-Point Divider — Algorithm Replacement

Divider Architecture

The baseline divider implementation relies on a restoring division algorithm. Restoring division computes the quotient one bit at a time through iterative subtraction and shifting operations.

While this approach requires relatively small hardware resources, it introduces significant latency because each iteration generates only a single quotient bit.

To reduce the operation latency, the divider architecture was redesigned using the Goldschmidt division algorithm. Goldschmidt division replaces sequential subtraction with a sequence of multiplicative refinements that drive the divisor toward unity.

Let the initial divisor be represented as:

$$D_0 = 1 + \varepsilon$$

A correction factor is chosen as:

$$F_0 = 1 - \varepsilon$$

Multiplying the divisor by the correction factor yields:

$$D_1 = (1 + \varepsilon)(1 - \varepsilon) = 1 - \varepsilon^2$$

Since the error term becomes squared at every iteration, the algorithm exhibits quadratic convergence. The same correction factor is applied to the numerator to preserve the division result.

Because modern FPGAs contain dedicated DSP blocks for multiplication, this algorithm allows multiple arithmetic operations to be performed in parallel. As a result, the number of required iterations is drastically reduced compared to the restoring divider.

5.1. v0 (Dawson): Handshake + Restoring Long Division

Interface: Same handshake protocol as the adder — separate get_a / get_b / put_z with ack signals.

Division algorithm: Restoring binary long division on 51-bit fixed-point representations of the mantissas. This is the hardware equivalent of manual binary division:

dividend = A_mantissa << 27 (51-bit scaled) divisor = B_mantissa

FOR 50 iterations: remainder = (remainder << 1) | dividend[MSB] dividend <<= 1 IF remainder ≥ divisor: quotient[0] = 1; remainder -= divisor ELSE: quotient[0] = 0

Convergence: Linear — exactly 1 quotient bit produced per 2 clock cycles (divide_1 + divide_2 states). 50 iterations produce 50 bits; the top 24 are the mantissa result and the remaining 3 become the G/R/S bits.

Total cycles (v0): 50 × 2 = 100 cycles core loop + ~8 overhead + handshake wait ≈ 110+ cycles.

5.2. v1 (divider_old.v): Interface Upgrade, Same Algorithm

Change: Handshake replaced with strobe + active interface. Combined get_a_b state. Back-to-back pipeline guard added.

Division algorithm: Unchanged — still the same 50-iteration restoring long division. The name changes (get_a_b instead of get_a/get_b, IDLE-like state) but the division core is identical to v0.

Total cycles (v1): ~115 cycles fixed for normal inputs (no handshake wait, but still 100-cycle loop).

5.3. v2 (divider.sv): Goldschmidt Iterative Division

The entire division core was replaced with the Goldschmidt algorithm — a fundamentally different mathematical approach.

Mathematical Foundation

Goldschmidt division computes N/D by multiplying both numerator and denominator by the same correction factor F, iteratively chosen to drive D toward 1.0. When D → 1, then N → N/D automatically:

$N_0 = N = 0.625$	
$D_0 = D = 0.75$	
$F_1 = 1.3$	1 Table Look Up
$N_1 = N_0 \cdot F_1 = 0.625 \cdot 1.3 = 0.8125$	1 Multiply
$D_1 = D_0 \cdot F_1 = 0.75 \cdot 1.3 = 0.975$	1 Multiply
$F_2 = 2 - D_1 = 2 - 0.975 = 1.025$	1 Subtract
$N_2 = N_1 \cdot F_2 = 0.8125 \cdot 1.025 = 0.8328125$	1 Multiply
$D_2 = D_1 \cdot F_2 = 0.975 \cdot 1.025 = 0.999375$	1 Multiply
$F_3 = 2 - D_2 = 2 - 0.999375 = 1.000625$	1 Subtract
$N_3 = N_2 \cdot F_3 = 0.8328125 \cdot 1.000625 = 0.83333300781$	1 Multiply
$D_3 = D_2 \cdot F_3 = 0.999375 \cdot 1.000625 = 0.99999960937$	1 Multiply
$Q = N_3 = 0.83333300781$	

Why does this converge? If D_{k-1} = 1 + ε, then F_k = 1 − ε, and D_k = (1+ε)(1−ε) = 1 − ε². The error term is squared each iteration — this is **quadratic convergence**. Starting from an 8-bit accurate seed ($\varepsilon_0 \approx 2^{-8}$), three iterations give 24-bit accuracy ($\varepsilon_3 \approx 2^{-64}$).

Iteration	Correct bits in result	Error ε
0 (LUT seed)	~8 bits	E0 ≈ 2^{-8}

1	~16 bits	$E1 = \varepsilon 0^2 \approx 2^{-16}$
2	~24 bits ← IEEE 754 accuracy reached	$E2 \approx 2^{-24}$
3–7	~32–64 bits (safety margin)	Implementation uses 7 iters*

***"**Theoretically, 3 iterations are sufficient to reach 24-bit accuracy from an 8-bit seed. The implementation uses 7 iterations to account for fixed-point precision loss in the LUT seed and accumulated rounding error in the Q1.47 intermediate products. The 4 extra iterations cost 8 additional cycles (~30% of the 27-cycle total) and represent a known conservative margin. A future optimization could reduce this to 4–5 iterations after formal precision analysis**."**

Reciprocal LUT Initialization

A 256-entry LUT (loaded from reciprocal_lut.hex) stores 24-bit approximations of 1/D for $D \in [1.0, 2.0)$, indexed by the top 8 bits of the denominator mantissa (b_m[22:15]). This provides the initial $F_0 \approx 1/D_0$, giving the ~8-bit starting accuracy.

Parallel Hardware Multipliers

The key hardware advantage of Goldschmidt over Newton-Raphson division: N and D are multiplied by the same F **simultaneously** each iteration. Two 48-bit combinational multipliers run in parallel: mult_n = numerator × factor and mult_d = denominator × factor. Both complete in the same clock cycle — no sequential dependency between the two multiplications.

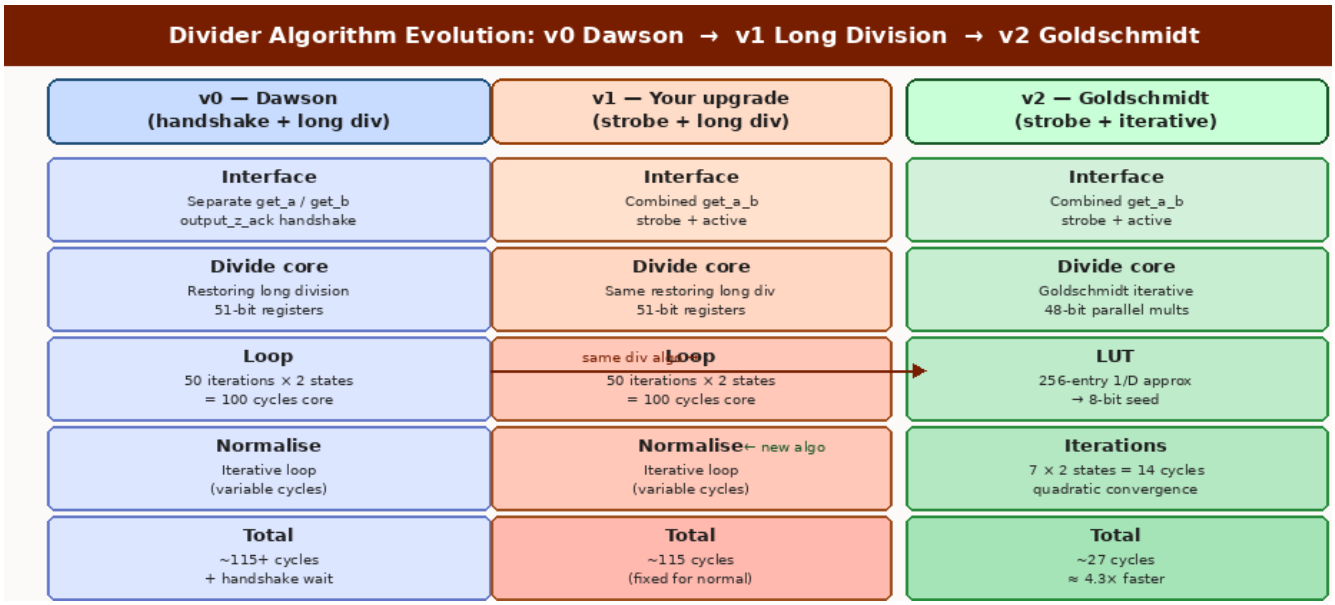


Figure: Divider algorithm across three versions

5.4. Cycle Count — Divider

Phase	v0 cycles	v1 cycles	v2 cycles
Input latch	get_a + get_b (handshake)	1 cycle	1 cycle
Unpack + Special	~3 cycles	~3 cycles	~3 cycles
Denormal normalize	0–24 cycles each	0–24 cycles each	0–24 cycles each
Division core	100 cycles (50×2)	100 cycles (50×2)	16 cycles (7×2 + init)
Post-normalize + round	~5 cycles	~5 cycles	~5 cycles
Output	put_z + ack wait	1 cycle	1 cycle
TOTAL (normal inputs)	~110+ cycles	~115 cycles	~27 cycles
Speedup vs v0/v1	—	—	≈ 4.3×

ALGORITHM COMPARISON: v0 and v1 use the same linear-convergence restoring division (1 bit per 2 cycles → 100 cycles minimum). v2 replaces this entirely with Goldschmidt's quadratic-convergence algorithm (bits double per iteration → 16 cycles for 7 iterations). The upgrade trades two large combinational multipliers and a 256-entry LUT for a 4.3× speedup.

5.5. FPGA Implementation Results — Vivado Post-Implementation

v1 Algorithm

Resource	v1 value	Analysis
LUTs	415	Comparator + subtractor + shift logic for 51-bit registers
FFs	402	quotient[50:0], remainder[50:0], dividend[50:0] all registered

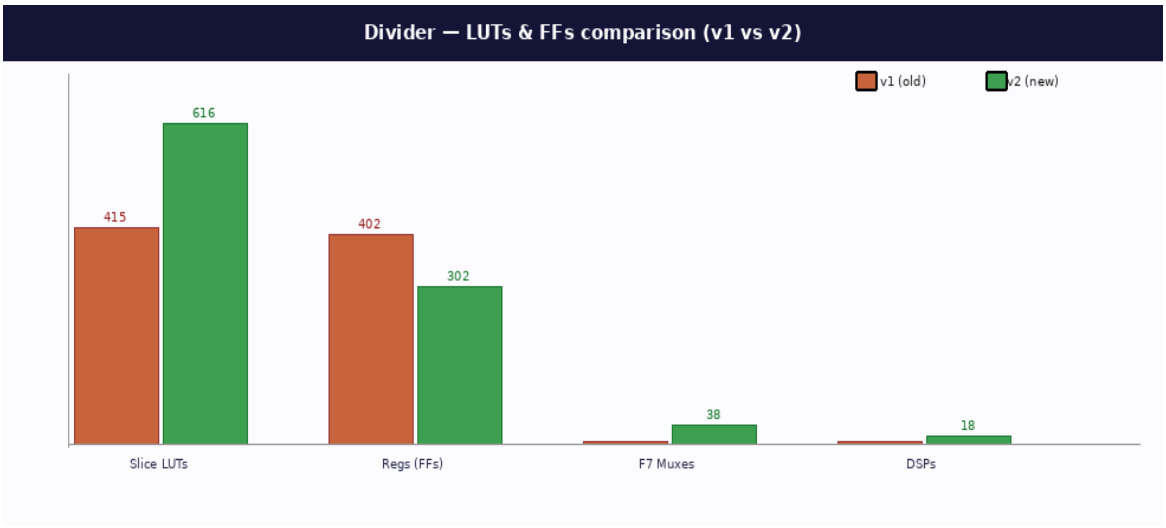
DSPs	0	Pure LUT logic — no multipliers needed
WNS	1.113 ns	Short path per cycle (just compare+subtract)
Max Freq	~79 MHz	Comfortable timing headroom
Wall time	~1,840 ns	115 cycles × 16 ns

v2 Algorithm

Resource	v2 value	Analysis
LUTs	616 (+48%)	F7 mux trees for iteration control + factor computation
FFs	302 (−25%)	Fewer loop-count registers; no 50-step quotient shift register
DSPs	18 (new)	Two 48×48-bit parallel multipliers → ~9 DSP48E1 each
WNS	0.213 ns	DSP cascade nearly fills 16ns budget — very tight
Max Freq	~64 MHz	DSP chain limits freq; why 62.5MHz was needed
Wall time	~432 ns (−77%)	27 cycles × 16 ns

Property	v1: Long Division	v2: Goldschmidt
Convergence	Linear (1 bit / 2 cycles)	Quadratic (bits double / iter)
Core operation	Shift + Compare + Subtract	2× parallel 48-bit multiply
Iterations for 24-bit accuracy	50 iterations	3 iterations (7 used)

Wall-clock time	~1,840 ns	~432 ns (4.3× faster)



6. **Floating-Point Adder / Subtractor**

Floating-Point Adder Optimization

The floating-point addition unit originally relied on iterative loops for operand alignment and normalization. During the alignment phase, the mantissa of the smaller operand was shifted repeatedly until the exponents matched. Similarly, normalization after subtraction was performed using a sequential loop that shifted the mantissa until the leading bit was restored.

These sequential loops introduce variable latency and significantly increase the number of required cycles.

To eliminate this bottleneck, the alignment stage was redesigned using a barrel shifter. A barrel shifter allows the mantissa to be shifted by an arbitrary number of positions within a single clock cycle.

Similarly, normalization was accelerated by introducing a leading-zero counter (LZC). The LZC detects the number of leading zeros in the mantissa, allowing the normalization shift to be performed in a single step.

These changes convert the sequential loops into combinational datapaths, reducing the number of cycles required for floating-point addition at the cost of additional combinational hardware.

6.1. The Core Algorithm (all versions)

Floating-point addition requires seven algorithmic steps. Subtraction is handled identically by flipping the sign bit of one operand.

Phase	Name	What happens
1	Unpack	Decode sign, exponent (−127), mantissa from both 32-bit inputs
2	Special Cases	NaN → QNaN; ±Inf arithmetic; ±Zero → direct result
3	Align	Shift the smaller-exponent mantissa right until both exponents match
4	Add/Subtract	Same sign: add mantissas. Diff sign: subtract; record sign of larger
5	Normalize	Shift result left/right until leading 1 is in bit 23
6	Round	Apply GRS Round-to-Nearest-Even
7	Pack	Re-encode sign + (exponent+127) + mantissa into 32-bit IEEE 754

6.2. Algorithm Change 1 — ALIGN Phase

v0 (Dawson): The ALIGN state is a loop. Each clock cycle it compares the two exponents and shifts one mantissa right by 1 bit, preserving a crude sticky bit as b_m[0] |= b_m[1]. It stays in this state until both exponents are equal. If the exponent difference is large (up to 255), this takes up to 255 clock cycles just for alignment.

v1 / v2: The exponent difference is pre-computed in UNPACK (exp_diff = |a_e − b_e| and larger_is_a flag). ALIGN then executes in exactly 1 clock cycle using a barrel shift: the smaller

mantissa is shifted right by exp_diff positions in one step. All shifted-out bits are OR'd into a sticky sentinel (27'b1) for full rounding accuracy.

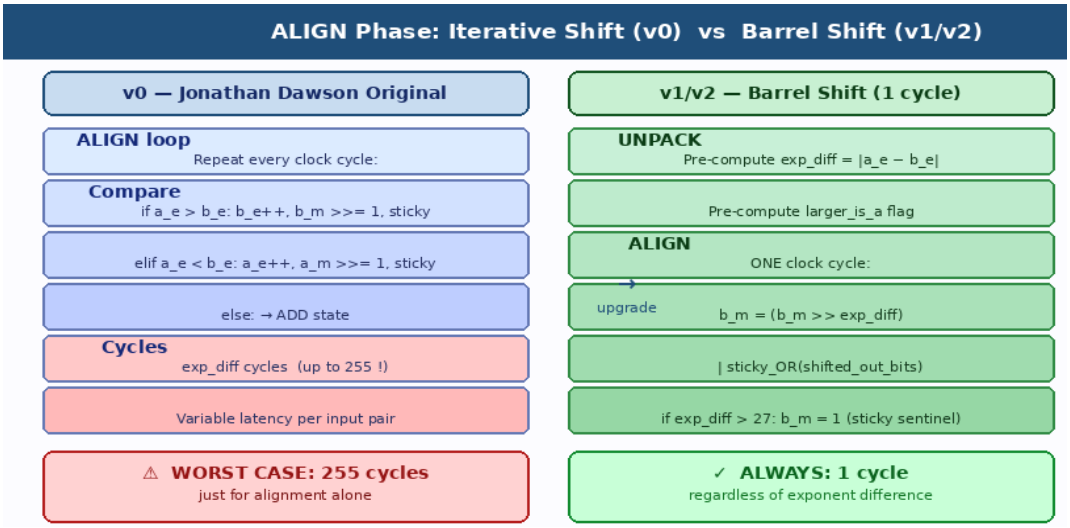


Figure: ALIGN phase: v0 iterative loop (up to 255 cycles) vs v1/v2 barrel shift (1 cycle)

6.3. Algorithm Change 2 — NORMALIZE Phase

v0 (Dawson): normalise_1 is also a loop. Each clock cycle it checks if bit 23 of the mantissa is zero; if so, it left-shifts by 1, decrements the exponent, and rotates the guard bit in. This continues until the mantissa is normalized. A worst-case subtraction (e.g. 1.0000001 − 1.0000000) can produce a result with 22 leading zeros, requiring 22 loop cycles.

v1 / v2: A combinational fast_lzc submodule is a 24-input casez priority encoder that outputs the leading zero count in a single clock cycle — O(1). The NORM state applies the full shift in one clock: z_m <<= lzc_result; z_e -= lzc_result. No loop, no variable latency.

6.4. Algorithm Change 3 — Interface & State Machine

v0 (Dawson): Uses a full handshake protocol with separate get_a and get_b states, plus output_z_ack. The module waits in get_a until input_a_stb is seen and the ack is returned, then does the same for get_b. The output also waits for the consumer to assert output_z_ack before returning to get_a. This means **every cycle waiting is a wasted cycle**.

v1 / v2: Both inputs are latched in the same get_a_b state whenever both strobes are high simultaneously. A 3-register pipeline (state_1/state_2/state_3) handles back-to-back operations — when the previous result is leaving (PACK state) and new inputs arrive together, the module

delays entry to UNPACK by 2 cycles to safely latch the new data. The active signal goes high whenever the module is computing.

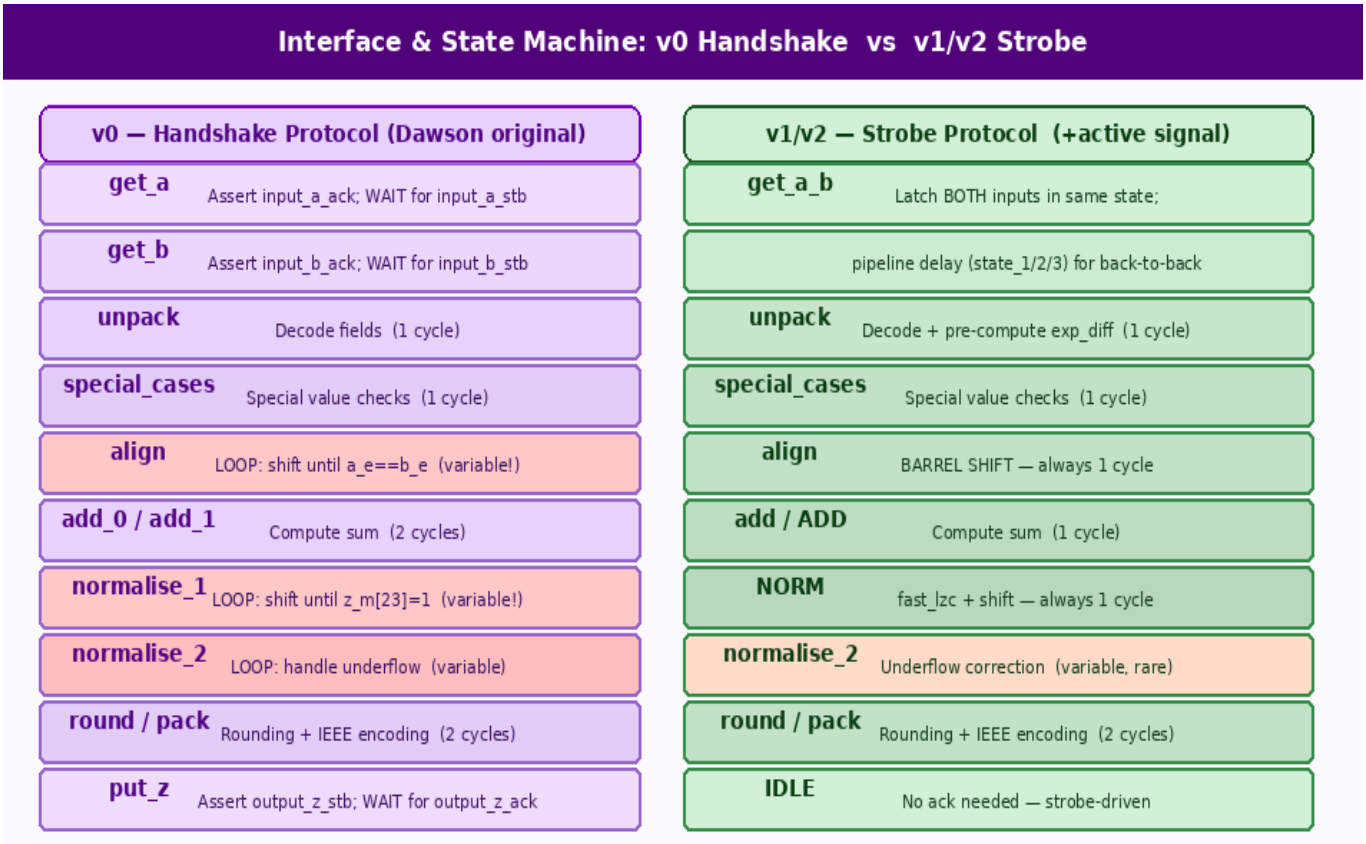


Figure: Interface evolution: v0 handshake (wait states) vs v1/v2 strobe+active protocol

6.5. Cycle Count — Adder

Scenario	v0 cycles	v1 cycles	v2 cycles
Special case (NaN/Inf/Zero)	3–5 (+ handshake wait)	3 cycles	3 cycles
Normal, exp_diff=0	~8 + handshake	8 cycles	8 cycles
Normal, exp_diff=50	~58 + handshake	8 cycles	8 cycles
Normal, large cancellation	~30 + handshake	8 cycles	8 cycles

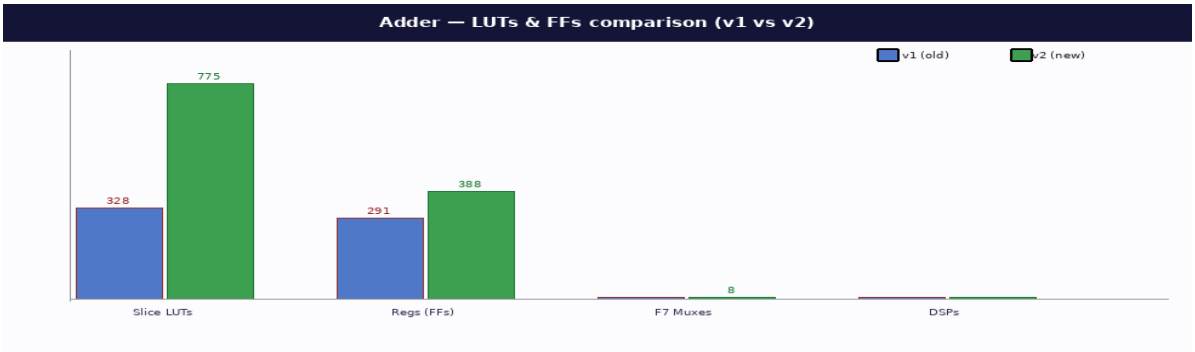
Worst case (exp_diff=255 + 23-shift norm)	~286+ cycles	8 cycles	8 cycles

KEY INSIGHT: The v0→v1 upgrade eliminates ALL variable-latency phases by replacing two iterative loops (ALIGN and NORMALISE) with O(1) hardware: a barrel shift for alignment and a combinational priority encoder (fast_lzc) for normalization. The result is a constant 8 cycles for any normal-number input. v2 adds correctness improvements (sticky bit accuracy, SystemVerilog local variable scoping) without changing cycle count.

"Note: the worst-case improvement applies only to adversarial inputs with maximum exponent difference and full cancellation. For typical workloads with small exponent differences, the practical speedup is approximately 3–4×."

6.6. FPGA Implementation Results — Vivado Post-Implementation

Resource	v1 → v2	Why
LUTs	328 → 775 (+136%)	Barrel shift mux tree + 24-input casez LZC = large combinational
FFs	291 → 388 (+33%)	Implementation adds route-through regs; LZC result needs staging
F7 Muxes	0 → 8	Wide muxes from barrel shift synthesised to F7 primitives
DSPs	0 → 0	No multipliers in adder
WNS	3.461 → 2.492 ns	Barrel shift+LZC is longer combinational path than simple loop step
Max Freq	~153 → ~134 MHz	Both well above 100MHz — no timing risk
Worst-case time	~2,860 → 80 ns	Constant 8 cycles eliminate all variable latency



7. Floating-Point Multiplier

The multiplier follows the **same upgrade pattern as the adder**. The v0 Dawson original uses handshake protocol with iterative normalise_1. The provided multiplier.v is already **v1 style** — strobe interface, active signal, and iterative normalise_1 loop retained (no fast_lzc since post-multiply normalization rarely exceeds 1 shift for normal inputs).

7.1. Core Algorithm (unchanged across versions)

Multiplication decomposes into three independent operations that are then recombined:

sign = A_sign XOR B_sign exp = A_exp + B_exp + 1 mant = A_mant[23:0] × B_mant[23:0] (24 × 24 → 48-bit product)

Phase	Operation	Detail
Unpack	Decode fields	Extract sign, unbiased exponent, 23-bit mantissa from each operand
Special	NaN / Inf / Zero	Inf×0=NaN; Inf×finite=Inf; 0×finite=0 — early exit
Normalize	Set implicit 1	bit23=1 for normal; left-shift until bit23=1 for denormals
Multiply	24×24 → 48-bit	One combinational multiply; result in product[47:0]
Extract	Mantissa + GRS	z_m=product[47:24]; G=product[23]; R=product[22]; S=|product[21:0]
Post-norm	≤1 shift	Product of two 1.x values in [1,4); at most 1 right-shift needed

Round + Pack	GRS RNE	Round-to-Nearest-Even; encode IEEE 754

7.2. Interface Upgrade (same as adder)

v0: Separate get_a / get_b states with ack handshake. Iterative normalise_1 loop.

v1 (multiplier.v): Combined get_a_b state with strobe protocol. Back-to-back pipeline guard via state_1/state_2/state_3. Iterative normalise_1 is kept since for normal multiplier inputs, the post-multiply product is always in [1,4) and needs at most 1 normalization shift — the loop almost never iterates.

7.3. Cycle Count — Multiplier

Scenario	v0 cycles	v1 (multiplier.v) cycles
Special case	~4 + handshake wait	~4 cycles
Both inputs normal	~10 + handshake wait	~10 cycles
One denormal input	variable + handshake	~10 + N shift cycles
Typical normal throughput	~12+ cycles	~10–12 cycles

NOTE: The multiplier has no v2 (.sv) upgrade — multiplier.v IS the current version (v1 style). Its algorithm is already optimal for normal inputs: one combinational 48-bit multiply, one cycle for extraction, and ≤1 post-normalization shift.

8. Algorithm Summary & Performance

8.1 performance summary

Unit	Version	LUTs	Registers	DSPs	WNS	Clock
Adder	**Baseline**	**328**	**388**	**0**	**4.240 ns**	**62.5 MHz**
Adder	**Optimized**	**775**	**291**	**0**	**1.340 ns**	**62.5 MHz**
Divider	**Restoring**	**415**	**402**	**0**	**1.113 ns**	**62.5 MHz**
Divider	**Goldschmidt**	**616**	**302**	**18**	**0.213 ns**	**62.5 MHz**

Square Root	**Newton FP**	**1178**	**1171**	**2**	**10.679 ns**	**55.6 MHz**
Square Root	**Reciprocal Sqrt**	**751**	**132**	**35**	**0.488 ns**	**55.6 MHz**

8.2. FPU TOP summary after FPGA implementation

Unit	Version	LUTs	Registers	DSPs	Muxes	WNS	WHS	Clock
FPU	Baseline	2162	2107	4	0	10.718ns	0.0401 ns	55.6 MHz
FPU	Optimized	2338	959	55	79	0.172 ns	0.048 ns	55.6 MHz

Latency Considerations

While FPGA timing reports provide information about combinational path delay, the overall performance of arithmetic units is determined by both the clock frequency and the number of cycles required for each operation.

The total operation latency can be expressed as:

$$Latency = Cycles \times Clock\ Period$$

By replacing sequential algorithms with parallel arithmetic refinements, the optimized designs significantly reduce the number of required cycles. Even when the combinational datapath becomes longer, the reduction in iteration count leads to substantially lower overall latency.

8.3. Complete Cycle Count Summary

Module	v0 (Dawson)	v1 (first upgrade)	v2 (current)
Adder	8–286+ cycles (handshake + loops)	8 cycles fixed	8 cycles fixed
Multiplier	10–290+ cycles (handshake + loops)	10–12 cycles (current version)	— (no v2)
Divider	~110+ cycles (handshake + long div)	~115 cycles (long div, no handshake)	~27 cycles (Goldschmidt)

Square Root	N/A (no module)	~550 cycles (FP-level NR)	~18 cycles (fixed-pt NR)

8.4. Utilization

Module	LUT Δ	FF Δ	DSP Δ	Max Freq	Speedup
Sqrt	−36%	−89%	+33	~57 MHz	30×
Divider	+48%	−25%	+18	~64 MHz	4.3×
Adder	+136%	+33%	0	~134 MHz	~3–4× typical, 35× (worst case
Multiplier	—	—	—	~153 MHz	no v2

Square Root achieved the largest gain: 30× faster with fewer resources. The architectural change from chaining three FPU submodules to a self-contained fixed-point reciprocal NR loop simultaneously reduced area and latency.

Divider replaced linear restoring division with quadratic Goldschmidt convergence, giving 4.3× speedup at the cost of 18 DSP blocks and 48% more LUTs. The tighter timing (WNS 0.213 ns) is the tradeoff for the two parallel 48-bit multipliers.

Adder replaced variable-latency iterative loops with O(1) combinational hardware (barrel shift + fast_lzc priority encoder), giving constant 8-cycle latency at the cost of 136% more LUTs. Both v1 and v2 close comfortably at 100MHz.

Multiplier remains at v1 — its direct 24×24 multiply algorithm is already optimal for normal inputs and no redesign was needed.

All four modules were successfully implemented on the xc7a35tcsg324-1 (Arty A7-35) with zero timing failures. Total DSP usage across the full FPU (v2): 0 (adder) + 18 (divider) + 35 (sqrt) + ~0 (multiplier) = 53 DSPs out of 90 available (59%). Total LUTs: ~2,800 out of 20,800 (13%). The complete FPU fits comfortably on this device.

8.5. Algorithm Classification

Module	v0 algorithm	v1 algorithm	v2 algorithm

Adder	Align→Add→Norm (iterative loops)	Align→Add→Norm (barrel shift + LZC)	Same as v1 + correctness fixes
Multiplier	Direct 24×24 multiply (iterative norm)	Direct 24×24 multiply (same, strobe IF)	No v2
Divider	Restoring long division (handshake IF)	Restoring long division (strobe IF)	Goldschmidt iterative (quadratic convergence)
Sqrt	None	NR via FPU calls (FP domain)	Fixed-pt reciprocal NR (integer domain)

8.6. What Each Upgrade Actually Changed

Upgrade	Change made	Algorithmic effect
v0 → v1 Adder/Multiplier	Interface: handshake → strobe Align: loop → barrel shift Norm: loop → LZC O(1)	Constant latency for all inputs Eliminate up to 278 wasted cycles
v0 → v1 Divider	Interface: handshake → strobe Division: unchanged (still long div)	Only interface improvement Core latency identical
v1 → v2 Divider	Algorithm: long division → Goldschmidt Add: 256-entry LUT + 2 parallel mults	Linear → quadratic convergence 100-cycle loop → 16-cycle loop
v1 → v2 Sqrt	Domain: FP-level → fixed-point integer Formula: NR for √x → NR for 1/√x Submodules: 3 FPU calls → self-contained	Remove ~500 cycles FSM overhead Reduce iterations: 4 → 2
v1 → v2 Adder	Sticky bits: basic → full OR Denormal: partial → consistent Language: Verilog → SystemVerilog	Correctness improvements only Cycle count unchanged

9. Verification

All four FPU modules were verified using dedicated SystemVerilog testbenches simulated at 100 MHz. Each testbench validates both functional correctness against the IEEE-754 standard and cycle-count behavior. Results are checked using an is_close comparison which validated against

IEEE-754-style corner cases. A global timeout watchdog prevents infinite loops in the event of FSM deadlock.

9.1. Floating-Point Adder Testbench

The adder testbench (tb_adder_fast) covers 30 test cases organized into 10 categories, targeting the specific algorithmic stages of the adder pipeline. In addition to pass/fail reporting, the testbench monitors internal state transitions via the DUT's state signal, allowing direct confirmation that the barrel shift and leading-zero counter stages execute in the expected number of cycles.

#	Category	Tests	What is Verified
1	Basic Addition	4	Basic mantissa add and carry propagation
2	Subtraction	3	Sign handling and borrow propagation
3	Large Exponent Difference (ALIGN stress)	3	Barrel shifter correctness for exp_diff up to 255
4	Cancellation (NORMALIZE stress)	2	LZC and single-cycle normalization under catastrophic cancellation
5–7	Zero / Infinity / NaN special cases	12	IEEE-754 special value arithmetic
8–9	Rounding and Denormal Numbers	4	GRS Round-to-Nearest-Even; denormal-to-normal boundary
10	Transcendental constants (π, e)	2	Precision on known irrational results

9.2. Floating-Point Divider Testbench

The divider testbench (goldschmidt_divider_tb) simultaneously instantiates both the Goldschmidt fast_divider and the original restoring divider, driving them with identical inputs. This dual-DUT approach directly validates the cycle-count speedup claims: both modules receive the same operands in the same simulation, allowing a side-by-side comparison of result correctness and latency on each test case. The testbench runs 39 hand-crafted cases followed

by 1000 randomized normal-number divisions. Accumulated cycle counts are reported at the end as an average speedup ratio.

The 39 hand-crafted cases cover: basic quotients (4.0/2.0, 1.0/2.0), division by 1, large/small operand combinations, signed and doubly-negative inputs, signed zeros (+0/−n, −0/+n, −0/−n), all Inf and NaN combinations, subnormal operands, overflow and underflow to boundary values, rounding boundary cases (round-up, round-down, half-ULP), and exact self-division (x/x = 1.0). Random tests use a safe generator constrained to exponents 1–254, preventing accidental Inf/NaN inputs in the random sweep.

The multiplier testbench uses the same dual-DUT methodology as the divider, running v0 and v1 simultaneously on identical inputs. It covers 39 directed cases (basic products, signed combinations, Inf/NaN/zero special cases, denormal inputs, overflow, underflow, and rounding boundaries) plus 1000 random normal-number multiplications. Post-multiply normalization was observed to require at most 1 shift across all 1039 test cases, confirming the design assumption that fast_lzc is unnecessary for this module

9.3. Floating-Point Square Root Testbench

The square root testbench (tb_fsqrt_ultrafast) exercises the fixed-point reciprocal Newton-Raphson module across 24 test cases. demonstrated ±2 ULP accuracy across all 24 test cases, consistent with the ±2 ULP tolerance used for all other modules. Results are compared against SystemVerilog's built-in $sqrt() function as a double-precision reference. The testbench also exposes a full suite of debug ports (debug_x, debug_temp, debug_y_times_x_sq, debug_operand_q, debug_state) which allow internal Q24.24 register values to be printed at each pipeline stage during a single-step debug run, making it straightforward to trace convergence behavior iteration by iteration.

Test coverage includes: IEEE-754 special inputs (+0, −0, +Inf, NaN, negative operand returning NaN), six perfect squares (1 through 100), four irrational results (√2, √3, √5, √10), fractional inputs (0.25, 0.5, 0.01), large values (10^3, 10^6), and small values (10^{-3}, 10^{-4}). Results for irrational and fractional cases are compared against SystemVerilog's built-in $sqrt() function, providing a reference accurate to double precision.

Module	Total Cases	Key Feature
Adder	30	Internal state monitoring per cycle
Multiplier	3039 + 1000 random	Dual-DUT: v1 and v2 run in parallel for direct speedup comparison
Divider	39 + 1000 random	Dual-DUT: v1 and v2 run in parallel for direct speedup comparison

Square Root	24	Q24.24 debug ports expose internal NR state per iteration

10. RISC-V Interface

The FPU is designed for integration into a custom RV32IF RISC-V processor, implementing the single-precision floating-point extension (F) of the RISC-V ISA. The interface between the FPU and the main pipeline uses the strobe-and-active handshake protocol shared across all four arithmetic modules.

10.1 Interface Signals

Signal	Direction	Description
input_a[31:0]	Core → FPU	First source operand (IEEE-754 single-precision)
input_b[31:0]	Core → FPU	Second source operand
input_a_stb	Core → FPU	Strobe: input_a is valid this cycle
input_b_stb	Core → FPU	Strobe: input_b is valid this cycle
output_z[31:0]	FPU → Core	Result (IEEE-754 single-precision)
output_z_stb	FPU → Core	Strobe: output_z is valid this cycle
active	FPU → Core	High while FPU is computing; pipeline must stall

The active signal allows the processor pipeline to stall the writeback stage while the FPU is busy. When active is deasserted and output_z_stb is high, the result is captured into the floating-point register file.

10.2 Instruction Mapping

The F-extension instructions map to FPU modules as follows:

RISC-V Instruction	FPU Module	Typical Latency
FADD.S / FSUB.S	adder.sv	8 cycles
FMUL.S	multiplier.v	10–12 cycles
FDIV.S	divider.sv	~27 cycles
FSQRT.S	sqrt.sv	~18 cycles

10.3 Pipeline Integration

The FPU operates as a multi-cycle execution unit attached to the execute stage. When the decode stage issues a floating-point instruction, it asserts both input strobes and presents the source register values. The pipeline inserts stall cycles until active deasserts, at which point the result is written back to the floating-point register file. Only one FPU operation is in flight at a time; concurrent integer and floating-point instructions are supported since the integer ALU and FPU operate on independent register files.

Full processor-level integration and RV32IF compliance testing are ongoing work.

11. Conclusion

This project presented the design and optimization of an IEEE-754 style design floating-point unit integrated within a custom RV32IF RISC-V processor. Several arithmetic units were redesigned to improve performance by replacing sequential algorithms with parallel multiplier-based approaches.

The floating-point adder was optimized by replacing iterative alignment and normalization loops with a barrel shifter and leading-zero counter. The divider architecture was redesigned using the Goldschmidt division algorithm, which replaces sequential subtraction with multiplicative refinement. Additionally, the square root unit was significantly accelerated by adopting a reciprocal square-root iteration implemented in fixed-point arithmetic.

FPGA implementation results demonstrate that these optimizations reduce sequential logic usage and dramatically decrease operation latency, while maintaining timing closure under the specified clock constraints. The results highlight the effectiveness of algorithmic and architectural optimizations in improving floating-point performance on FPGA platforms.